

Trabajo Práctico Integrador

Red Neuronal con Aplicación Web

Materia: Redes Neuronales Profundas - Ingeniería en Sistemas de Información

Objetivo del Trabajo Práctico Integrador

El objetivo de este trabajo es que integren los conocimientos adquiridos sobre redes neuronales desarrollando una aplicación web que incluya un componente inteligente que hayan entrenado ustedes mismos. Con este trabajo, experimentarán con cada una de las etapas del ciclo de vida de un modelo de red neuronal, desde la selección del dataset y el preprocesamiento de los datos hasta la implementación y despliegue de una aplicación web funcional.

Este año el trabajo se enfocará exclusivamente en problemas de visión por computadora utilizando imágenes.

Consigna

Cada grupo (máximo de 5 estudiantes) deberá desarrollar una aplicación web que cumpla con las siguientes características:

1. **Componente Inteligente:** desarrollen y entrenen una red neuronal que resuelva un problema específico de visión por computadora usando un dataset de imágenes de su elección. El problema puede ser de clasificación, detección, segmentación, generación, u otra aplicación relevante sobre imágenes. Como mínimo, deberán realizar un fine-tuning de un modelo preentrenado. No se permite el uso directo de un modelo preentrenado sin modificaciones.
2. **Dataset:** seleccionen un dataset público de imágenes que consideren relevante para el problema. Pueden explorar sitios como [Kaggle](#), [UCI Machine Learning Repository](#), o cualquier fuente de datos abierta. El dataset debe ser lo suficientemente interesante como para justificar el uso de una red neuronal, pero no tan complejo como para exceder las capacidades computacionales disponibles.
3. **Entrenamiento de la Red Neuronal:** desarrollen y entrenen la red neuronal utilizando PyTorch. Documenten su proceso de selección del modelo, ajuste de hiperparámetros y evaluación del rendimiento. Se aceptan fine-tuning de modelos preentrenados, pero no la utilización directa de los mismos.
4. **Aplicación Web:** implementen una aplicación web simple que permita interactuar con el modelo entrenado. Se recomienda enfáticamente usar Streamlit por su simplicidad. La interfaz debe ser funcional, sin necesidad de un diseño visual avanzado o de UI/UX complejo.
5. **Hosting:** la aplicación debe ser desplegada en un servicio gratuito de hosting. Recomendamos usar [Streamlit Cloud](#) para el despliegue. Estas plataformas permiten el hosting gratuito de aplicaciones pequeñas y son adecuadas para principiantes.

Organización de los horarios de presentación

Para todas las presentaciones (las cuatro entregas semanales y la presentación final), los horarios de exposición de cada grupo se asignarán por sorteo en clase. Los grupos que necesiten cambiar su horario asignado deberán negociarlo directamente con otro grupo que esté dispuesto a intercambiar. El profesor no reasigna horarios unilateralmente.

Entregables

1. Primera Semana — Elección del dataset y definición de la aplicación

Entregable: cada grupo presentará oralmente **tres opciones** de dataset/aplicación. El profesor elegirá la más adecuada considerando los tiempos del proyecto.

Como soporte para la exposición, el grupo debe traer una presentación de PowerPoint o un notebook que acompañe lo que cuentan. No se entrega ni se evalúa un documento escrito: se evalúa lo que cada integrante comunica oralmente sobre el trabajo realizado.

La nota es individual, no grupal. Cada integrante debe poder explicar y defender oralmente cualquier parte del trabajo del grupo, no solo la que le tocó. Se espera que todos hayan explorado los datasets y participado en la decisión. Esta regla aplica a todas las entregas del proyecto.

Para cada una de las tres opciones, la exposición debe cubrir:

a) Descripción del problema y de la aplicación propuesta

- Qué problema concreto resuelve la aplicación (clasificación, detección, segmentación, etc.).
- Cómo sería la interacción en la web app: qué ingresa el usuario, qué devuelve el modelo.
- Justificación de por qué una red neuronal es adecuada para este problema (vs. métodos más simples).

b) Análisis del dataset

- Fuente, licencia y enlace.
- Cantidad total de ejemplos y distribución por clase (incluir un gráfico o tabla si hay desbalance).
- Tamaño en disco y formato de los archivos.
- Resolución, relación de aspecto, condiciones de captura (iluminación, fondo, variabilidad), presencia de ruido o artefactos en las imágenes.
- **Mostrar al menos 6 a 10 ejemplos reales del dataset** y comentar la calidad observada.
- Cómo está particionado (o cómo se piensa particionar) en train/val/test.

Fechas de entrega: miércoles 27, jueves 28 o sábado 30 de mayo de 2026.

2. Segunda Semana — Creación del dataset en PyTorch

Entregable: cada grupo presentará oralmente el trabajo de preparación del dataset, acompañado de un notebook de Jupyter que muestre el código y los resultados. Se evalúa lo que cada integrante comunica oralmente.

Recordatorio: la nota es individual. Cada integrante debe poder explicar y defender cualquier parte del trabajo realizado.

La exposición y el notebook deben cubrir:

a) Carga y organización del dataset

- Estructura de carpetas / archivos del dataset original y cómo se accede a las imágenes y sus etiquetas.
- Implementación de una clase Dataset de PyTorch (heredando de `torch.utils.data.Dataset`) o uso justificado de `ImageFolder` si la estructura lo permite.

- Implementación de los DataLoader para train, validación y test, con batch_size y shuffle adecuados.

b) Particionado

- Cómo se dividió el dataset en train / validación / test, con cantidad de imágenes y distribución por clase en cada partición.
- Justificación del criterio de partición (por ejemplo, estratificado para mantener balance de clases, o separación por sujeto si las imágenes están agrupadas).
- Semilla (seed) fijada para que la partición sea reproducible.

c) Preprocesamiento

- Redimensionado al tamaño que espera el modelo preentrenado elegido (típicamente 224x224 para modelos tipo ResNet/EfficientNet).
- Normalización con los valores de media y desvío correspondientes al modelo preentrenado.
- Conversión a tensor y cualquier otro paso necesario (conversión de canales, manejo de imágenes en escala de grises, etc.).

d) Data augmentation

- Qué transformaciones se aplicaron al conjunto de entrenamiento y por qué tienen sentido para este problema en particular.
- Confirmación de que las augmentations **solo se aplican a train**, no a validación ni test.
- **Mostrar visualmente** el efecto de las augmentations: tomar 4–6 imágenes y mostrar varias versiones aumentadas de cada una.

e) Verificación final

- Mostrar un batch del DataLoader de train ya procesado (imágenes desnormalizadas para visualización + etiquetas correspondientes), confirmando que los pares imagen-etiqueta son correctos.
- Reportar dimensiones de los tensores (batch_size, canales, alto, ancho) y rango de valores tras la normalización.

f) Repositorio de GitHub

A partir de esta entrega, el trabajo debe estar en un repositorio de GitHub (la estructura final está definida en la sección Conformación del Repositorio). Para esta semana se espera específicamente:

- Repo creado con la estructura de carpetas data/, dev/ y prod/ (las dos últimas pueden estar vacías o con placeholders).
- El notebook de preparación de datos ubicado en dev/ con un nombre descriptivo (por ejemplo, 01_dataset_preparation.ipynb).
- **Las imágenes del dataset NO se suben al repositorio.** Git no está pensado para archivos binarios grandes y GitHub tiene límites estrictos (100 MB por archivo). Configurar un .gitignore desde el inicio que excluya las carpetas de imágenes. Por ejemplo:

```
data/raw/
data/processed/
*.jpg
*.jpeg
*.png
```
- La carpeta data/ del repo debe contener:

- Un README .md que explique de dónde sale el dataset, link a la fuente original (Kaggle, etc.) y cómo descargarlo.
- Un script o celda en el notebook que **descargue y descomprima automáticamente** el dataset en la estructura esperada.
- Los CSV de splits (train.csv, val.csv, test.csv) con las rutas relativas de cada imagen y su etiqueta. Estos archivos sí se versionan porque son livianos y aseguran que todos trabajen con el mismo particionado.
- **Reproducibilidad:** el notebook debe poder correrse de punta a punta en una máquina nueva siguiendo estos pasos: clonar repo → instalar dependencias → correr script de descarga → correr notebook. Sin pasos manuales intermedios.

Fechas de entrega: miércoles 3, jueves 4 o sábado 6 de junio de 2026.

3. Tercera Semana — Entrenamiento y selección del mejor modelo

Entregable: cada grupo presentará oralmente el proceso de entrenamiento y la justificación del modelo elegido, acompañado de un notebook de Jupyter con el código, los experimentos y los resultados. Se evalúa lo que cada integrante comunica oralmente.

Recordatorio: la nota es individual. Cada integrante debe poder explicar y defender cualquier parte del trabajo realizado, incluyendo decisiones técnicas sobre arquitectura, hiperparámetros y métricas.

La exposición y el notebook deben cubrir:

a) Modelo preentrenado y estrategia de fine-tuning

- Modelo preentrenado elegido (ResNet, EfficientNet, ViT, etc.) y justificación de la elección.
- Modificaciones realizadas a la arquitectura: capa final reemplazada para la cantidad de clases del problema, capas adicionales si se agregaron, etc.
- Estrategia de fine-tuning aplicada: ¿se congelaron capas? ¿Cuáles? ¿Se entrena todo end-to-end? Justificar la decisión en función del tamaño del dataset y la similitud con el dominio original del modelo.

b) Configuración del entrenamiento

- Función de pérdida elegida y por qué (cross-entropy, focal loss si hay desbalance fuerte, etc.).
- Optimizador (SGD, Adam, AdamW) y learning rate inicial.
- Scheduler de learning rate si se usó (step, cosine, reduce on plateau).
- Cantidad de épocas, batch size, y cualquier otra configuración relevante.
- Hardware utilizado (Colab, GPU local, etc.) y tiempo aproximado de entrenamiento.

c) Experimentación

- Se espera que el grupo haya probado **al menos 3 configuraciones distintas** y las compare. Las variantes pueden ser: distintos modelos preentrenados, distintas estrategias de fine-tuning (más o menos capas congeladas), distintos learning rates, distintas augmentations, etc.
- Tabla comparativa de los experimentos con las métricas obtenidas en validación.
- Justificar qué se aprendió de cada experimento y cómo guió la decisión siguiente.

d) Evaluación del modelo elegido

- Curvas de pérdida y métrica principal (accuracy, F1, etc.) en train y validación a lo largo de las épocas. Comentar si hay overfitting o underfitting y cómo se manejó.

- Métricas finales **sobre el conjunto de test** (no validación). Para clasificación: accuracy, precision, recall y F1 por clase, más matriz de confusión.
- Análisis de errores: mostrar algunos ejemplos mal clasificados y comentar posibles causas (clases visualmente similares, imágenes de baja calidad, sesgos del dataset, etc.).

e) Guardado del modelo

- Los pesos del modelo final entrenado deben guardarse como `modelo.pth` dentro de la carpeta `dev/` del repositorio. Si también quieren guardar pesos de experimentos intermedios, también van en `dev/`.
- Tener en cuenta que los archivos `.pth` pueden superar el límite de 100 MB de GitHub. En ese caso, considerar el uso de **Git LFS** (Large File Storage).

f) Repositorio

- El notebook de esta semana debe estar en `dev/` con un nombre descriptivo (por ejemplo, `02_model_training.ipynb`).
- Los experimentos descartados también quedan documentados en el notebook (no hace falta un notebook por experimento; alcanza con secciones bien delimitadas en uno solo).

Fechas de entrega: miércoles 10, jueves 11 o sábado 13 de junio de 2026.

4. Cuarta Semana — Interfaz y despliegue

Entregable: cada grupo presentará oralmente la aplicación desplegada y funcionando, mostrándola en vivo. El repositorio de GitHub debe estar completo según la estructura definida en Conformación del Repositorio, y la URL de la aplicación desplegada debe ser accesible públicamente. Se evalúa lo que cada integrante comunica oralmente.

Recordatorio: la nota es individual. Cada integrante debe poder explicar y defender cualquier parte del trabajo realizado, incluyendo el código de la app, la carga del modelo y el flujo de inferencia.

La exposición debe cubrir:

a) Aplicación web

- Demostración en vivo de la app funcionando: el usuario sube una imagen (o usa la cámara, según el caso), la app procesa la imagen y devuelve la predicción del modelo.
- Mostrar qué pasa con casos esperados (imágenes representativas del dataset) y con casos límite (imágenes que no corresponden a ninguna clase, imágenes de baja calidad, etc.). Comentar cómo se comporta el modelo en cada caso.
- La interfaz debe ser **funcional y clara**, no se evalúa diseño visual avanzado. Se valora que muestre la información útil: imagen ingresada, clase predicha y, si corresponde, las probabilidades por clase.
- **Tecnología de la interfaz:** se recomienda enfáticamente usar **Streamlit**. Es la tecnología pensada para este proyecto, la más simple para integrar un modelo de PyTorch, y la que mejor se ajusta a los tiempos disponibles. Excepcionalmente, los grupos a quienes les resulte muy molesto trabajar con Streamlit pueden usar otra tecnología (React, Next.js, Gradio, Flask + HTML, etc.), siempre y cuando se cumplan **todos** los requerimientos de este enunciado sin excepción. El uso de una tecnología alternativa no es motivo para flexibilizar ningún otro requisito.

b) Código de la aplicación

- Estructura del código separada en `app.py` (interfaz) y `utils.py` (funciones auxiliares: carga del modelo, preprocesamiento de la imagen de entrada, postprocesamiento de la salida). Los grupos que excepcionalmente usen otra tecnología deben adaptar la organización pero mantener la misma separación de responsabilidades.
- **El preprocesamiento de la imagen de entrada en producción debe ser idéntico al usado en validación/test durante el entrenamiento** (mismo `resize`, misma normalización). Este es uno de los errores más comunes y rompe silenciosamente el modelo.
- Carga del modelo cacheada con `@st.cache_resource` (u equivalente en la tecnología elegida) para que no se recargue en cada interacción del usuario.

c) Despliegue

- App desplegada y accesible públicamente desde una URL, sin requerir login ni instalación. La URL debe estar en el `README.md` del repositorio.
- Para Streamlit, el hosting es en **Streamlit Cloud** (gratuito y directamente integrado con GitHub). Los grupos que excepcionalmente usen otra tecnología deberán buscar un servicio de hosting gratuito apropiado y garantizar la accesibilidad pública.
- El `requirements.txt` debe estar completo y con versiones fijadas (por ejemplo, `torch==2.x.x`, no solo `torch`) para que el despliegue sea reproducible.
- Si el archivo `modelo.pth` supera el límite de tamaño del servicio de hosting, documentar la estrategia usada para cargarlo (descarga desde Drive/Hugging Face al iniciar la app, Git LFS, etc.).

d) Repositorio final

- Estructura completa según Conformación del Repositorio: carpetas `data/`, `dev/` y `prod/` con todos los archivos correspondientes.
- `README.md` en la raíz del repo con: descripción del proyecto, integrantes, link a la app desplegada, instrucciones para clonar y correr localmente, y link al dataset.
- Repo público y URL compartida con el profesor.

Fechas de entrega: miércoles 17, jueves 18 o sábado 20 de junio de 2026.

5. Presentación Final

Cada grupo realizará una **presentación oral de 15 minutos como máximo** que integre todo el trabajo de las cuatro semanas, seguida de una ronda de preguntas. **Se controlará el tiempo durante la exposición.** La presentación se apoya en un PowerPoint (u otra herramienta de slides) y debe incluir una demostración en vivo de la app desplegada.

Recordatorio: la nota es individual. Cada integrante debe poder explicar y defender cualquier parte del proyecto, no solamente la sección que le tocó exponer. Las preguntas posteriores a la presentación pueden dirigirse a cualquier integrante sobre cualquier tema del trabajo.

a) Contenido de la presentación

La presentación debe cubrir las cuatro etapas del proyecto de forma integrada:

- **Problema y dataset:** qué resuelve la aplicación, qué dataset se eligió y por qué. Mostrar ejemplos representativos del dataset.
- **Preparación de datos:** cómo se preprocesaron las imágenes, qué augmentations se aplicaron y por qué, cómo se particionó el dataset.
- **Entrenamiento:** modelo preentrenado elegido, estrategia de fine-tuning, experimentos realizados, métricas finales en test y análisis de errores.

- **Aplicación y despliegue:** arquitectura de la app, decisiones de implementación, demo en vivo.

b) Demostración en vivo

- La demo debe correrse preferentemente contra la **app desplegada online**, como prueba de que el despliegue funciona.
- Si por algún motivo la app online no responde durante la presentación (caída del servicio, problemas de red, etc.), se acepta correr la app en local para mostrar el funcionamiento. Lo que no se acepta es no mostrarla funcionando.
- Tener preparado un set de imágenes de prueba (representativas + casos límite) para mostrar durante la demo.

c) Distribución del tiempo

Como referencia, sugerimos repartir los 15 minutos aproximadamente así:

- Problema y dataset: 3 min
- Preparación de datos: 2 min
- Entrenamiento y resultados: 5 min
- Aplicación y demo en vivo: 4 min
- Cierre y conclusiones: 1 min

d) Participación de los integrantes

- Todos los integrantes del grupo deben exponer durante la presentación. La distribución de qué expone cada uno es libre, pero ningún integrante puede quedar sin hablar.
- Durante las preguntas, el profesor puede dirigirse a cualquier integrante sobre cualquier tema del proyecto, independientemente de qué sección expuso.

e) Qué se evalúa

- Claridad y profundidad técnica de la exposición de cada integrante.
- Capacidad de justificar decisiones tomadas durante el proyecto (por qué este dataset, por qué este modelo, por qué estas augmentations, etc.).
- Capacidad de responder preguntas sobre cualquier parte del trabajo.
- Funcionamiento efectivo de la app durante la demo.

Fechas de presentación: miércoles 24, jueves 25 o sábado 27 de junio de 2026.

Conformación del Repositorio

El repositorio de GitHub del proyecto debe seguir la siguiente estructura:

6. **data/**: contiene el README .md del dataset, el script de descarga y los CSV de splits. **Las imágenes en sí no se versionan** (ver detalle en la segunda entrega).
7. **dev/**: contiene los notebooks de desarrollo experimental (preparación de datos, entrenamiento, evaluación) y el archivo `modelo.py` con los pesos del modelo final.
8. **prod/**: contiene los archivos necesarios para correr la aplicación:
 - **app.py**: archivo principal de la aplicación web (interfaz Streamlit).
 - **utils.py**: funciones auxiliares (carga del modelo, preprocesamiento, postprocesamiento).
 - **requirements.txt**: dependencias necesarias para ejecutar el proyecto, con versiones fijadas.

- **README.md**: descripción del proyecto, integrantes, link a la app desplegada e instrucciones de uso.

Los grupos que excepcionalmente usen una tecnología distinta a Streamlit deben adaptar el contenido de prod/ manteniendo la separación entre interfaz, lógica auxiliar y dependencias.

Ejemplo guía

Se adjuntan enlaces a ejemplos de años anteriores como este [generador de gatitos](#) y este [detector de letras en lenguaje de señas](#).